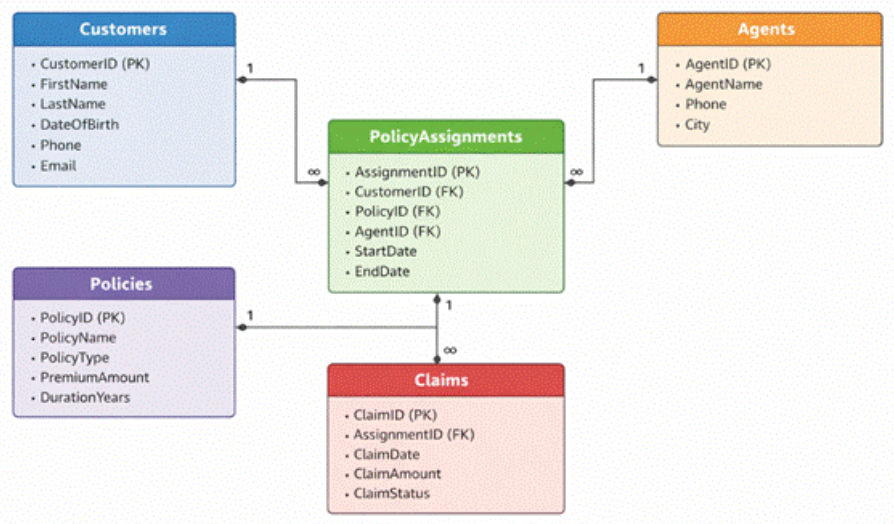
DAY 4.4 Practical project assignment

NAME: A.SANJAY KUMAR

Email : attellisanjay29@gmail.com



Create Database command

```
CREATE DATABASE insurance_db;
Use insurance_db;
```

Create table commands for all tables

```
CREATE TABLE insurance_customers (
    customer_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    dob DATE,
    phone VARCHAR(15),
    email VARCHAR(100),
    created_at DATE,
    updated_at DATE
);

CREATE TABLE policies (
    policy_id INT PRIMARY KEY,
    policy_name VARCHAR(50),
    policy_type VARCHAR(50),
    pre_amount INT,
    duration_year INT,
    created_at DATE,
    updated_at DATE
);


CREATE TABLE agents (
    agent_id INT PRIMARY KEY,
```

```
    agent_name VARCHAR(100),
    phone VARCHAR(15),
    city VARCHAR(50),
    created_at DATE,
    updated_at DATE
);


CREATE TABLE policy_assignments (
    assignment_id INT PRIMARY KEY,
    customer_id INT,
    policy_id INT,
    agent_id INT,
    start_date DATE,
    end_date DATE,
    created_at DATE,
    updated_at DATE,
    FOREIGN KEY (customer_id) REFERENCES insurance_customers(customer_id),
    FOREIGN KEY (policy_id) REFERENCES policies(policy_id),
    FOREIGN KEY (agent_id) REFERENCES agents(agent_id)
);


CREATE TABLE claims (
    claim_id INT PRIMARY KEY,
    assignment_id INT,
    claim_date DATE,
    claim_amount INT,
    claim_status VARCHAR(30),
    created_at DATE,
    updated_at DATE,
    FOREIGN KEY (assignment_id) REFERENCES policy_assignments(assignment_id)
);
```

Insert commands for all tables

Data Inserted in to insurance_customers Table:

```
INSERT INTO insurance_customers VALUES
(1,'Sanjay','Attelli','2005-01-29','8919200290','sanjay.attelli@gmail.com','2024-01-05','2024-06-10'),
(2,'Manoj','Kumar','2004-05-25','9123456789','manoj.kumar@yahoo.com','2024-02-12','2024-07-01'),
(3,'Amit','Sharma','1998-04-12','9000000001','amit.sharma@gmail.com','2024-03-01','2024-06-01'),
(4,'Priya','Singh','1996-09-21','9000000002','priya.singh@gmail.com','2024-03-02','2024-06-02'),
(5,'Rahul','Verma','1994-01-15','9000000003','rahul.verma@yahoo.com','2024-03-03','2024-06-03'),
(6,'Neha','Gupta','2000-11-30','9000000004','neha.gupta@gmail.com','2024-03-04','2024-06-04'),
(7,'Vikas','Reddy','1992-07-18','9000000005','vikas.reddy@gmail.com','2024-03-05','2024-06-05'),
(8,'Pooja','Nair','1999-02-25','9000000006','pooja.nair@gmail.com','2024-03-06','2024-06-06'),
(9,'Arjun','Patel','1991-12-09','9000000007','arjun.patel@yahoo.com','2024-03-07','2024-06-07'),
(10,'Kiran','Das','1997-06-14','9000000008','kiran.das@gmail.com','2024-03-08','2024-06-08'),
```

(11,'Sneha','Joshi','1995-10-10','9000000009','sneha.joshi@gmail.com','2024-03-09','2024-06-09'),
(12,'Rohit','Mehta','1993-05-05','9000000010','rohit.mehta@gmail.com','2024-03-10','2024-06-10');

Data Inserted in to PoliciesTable:

INSERT INTO policies VALUES
(101,'Life Secure Plan','Life Insurance',12000,20,'2023-12-01','2024-05-01'),
(102,'Health Plus Policy','Health Insurance',8500,1,'2024-01-15','2024-06-20'),
(103,'Child Secure','Life Insurance',15000,18,'2024-02-01','2024-06-01'),
(104,'Senior Health','Health Insurance',22000,1,'2024-02-02','2024-06-02'),
(105,'Family Health Plus','Health Insurance',18000,2,'2024-02-03','2024-06-03'),
(106,'Term Life Gold','Life Insurance',25000,30,'2024-02-04','2024-06-04'),
(107,'Personal Accident','General Insurance',6000,1,'2024-02-05','2024-06-05'),
(108,'Travel Secure','Travel Insurance',4000,1,'2024-02-06','2024-06-06'),
(109,'Home Safe','Property Insurance',12000,10,'2024-02-07','2024-06-07'),
(110,'Vehicle Protect','Motor Insurance',9000,1,'2024-02-08','2024-06-08'),
(111,'Health Premium','Health Insurance',30000,3,'2024-02-09','2024-06-09'),
(112,'Life Platinum','Life Insurance',35000,25,'2024-02-10','2024-06-10');

Data Inserted in to agents Table:

INSERT INTO agents VALUES
(5001,'Sanjay Kumar','8919200290','Hyderabad','2023-11-10','2024-04-12'),
(5002,'Monoj Kumar','9123456781','Delhi','2023-10-05','2024-03-18'),
(5003,'Varshith','9988776651','Chennai','2023-09-20','2024-02-25'),
(5004,'Amit Rao','9011111111','Mumbai','2023-08-01','2024-04-01'),
(5005,'Priya Shah','9011111112','Pune','2023-08-02','2024-04-02'),
(5006,'Rakesh Jain','9011111113','Delhi','2023-08-03','2024-04-03'),
(5007,'Sunita Roy','9011111114','Kolkata','2023-08-04','2024-04-04'),
(5008,'Deepak Mishra','9011111115','Bhopal','2023-08-05','2024-04-05'),
(5009,'Anil Kumar','9011111116','Chennai','2023-08-06','2024-04-06'),
(5010,'Meena Iyer','9011111117','Coimbatore','2023-08-07','2024-04-07'),
(5011,'Suresh Naik','9011111118','Goa','2023-08-08','2024-04-08'),
(5012,'Kavya Shetty','9011111119','Mangalore','2023-08-09','2024-04-09'),
(5013,'Nitin Kulkarni','9011111120','Nagpur','2023-08-10','2024-04-10');

Data Inserted in to policy_assignments Table:

INSERT INTO policy_assignments VALUES
(1,1,101,5001,'2024-01-01','2044-01-01','2024-01-02','2024-06-01'),
(2,2,102,5002,'2024-06-15','2025-06-15','2024-06-16','2024-07-10'),
(3,3,103,5004,'2024-01-10','2042-01-10','2024-01-11','2024-06-01'),
(4,4,104,5005,'2024-02-01','2025-02-01','2024-02-02','2024-06-02'),
(5,5,105,5006,'2024-03-01','2026-03-01','2024-03-02','2024-06-03'),
(6,6,106,5007,'2024-04-01','2054-04-01','2024-04-02','2024-06-04'),
(7,7,107,5008,'2024-05-01','2025-05-01','2024-05-02','2024-06-05'),
(8,8,108,5009,'2024-06-01','2025-06-01','2024-06-02','2024-06-06'),
(9,9,109,5010,'2024-07-01','2034-07-01','2024-07-02','2024-06-07'),
(10,10,110,5011,'2024-08-01','2025-08-01','2024-08-02','2024-06-08'),
(11,11,111,5012,'2024-09-01','2027-09-01','2024-09-02','2024-06-09'),
(12,12,112,5013,'2024-10-01','2049-10-01','2024-10-02','2024-06-10');

Data Inserted in to claims Table:

```
INSERT INTO claims VALUES
(1,1,'2024-03-10',50000,'Approved','2024-03-11','2024-03-20'),
(2,2,'2024-08-05',12000,'Pending','2024-08-06','2024-08-15'),
(3,3,'2024-04-15',20000,'Approved','2024-04-16','2024-04-20'),
(4,4,'2024-05-10',15000,'Pending','2024-05-11','2024-05-18'),
(5,5,'2024-06-05',30000,'Rejected','2024-06-06','2024-06-12'),
(6,6,'2024-07-01',50000,'Approved','2024-07-02','2024-07-10'),
(7,7,'2024-08-12',8000,'Pending','2024-08-13','2024-08-20'),
(8,8,'2024-09-18',12000,'Approved','2024-09-19','2024-09-25'),
(9,9,'2024-10-22',25000,'Approved','2024-10-23','2024-10-30'),
(10,10,'2024-11-05',9000,'Rejected','2024-11-06','2024-11-12'),
(11,11,'2024-11-20',40000,'Pending','2024-11-21','2024-11-28'),
(12,12,'2024-12-01',60000,'Approved','2024-12-02','2024-12-10');
```

Queries and answer:

SELECT

1. Display all customer details

```
SELECT * FROM insurance_customers;
```

2. Display all policy details

```
SELECT * FROM policies;
```

3. Display all agent details

```
SELECT * FROM agents;
```

WHERE

1. Display customers from city Delhi

```
SELECT * FROM insurance_customers
WHERE city = 'Delhi';
```

2. Display policies with premium greater than 10000

```
SELECT * FROM policies
WHERE pre_amount > 10000;
```

3. Display rejected claims

```
SELECT * FROM claims
WHERE claim_status = 'Rejected';
```

ORDER BY

1. Display customers ordered by first name

```
SELECT * FROM insurance_customers
ORDER BY first_name;
```

2. Display policies ordered by premium descending

```
SELECT * FROM policies
ORDER BY pre_amount DESC;
```

3. Display claims ordered by claim date

```
SELECT * FROM claims
ORDER BY claim_date;
```

STRING FUNCTIONS

1. Display customer names in uppercase

```
SELECT UPPER(first_name) FROM insurance_customers;
```

2. Display trimmed agent names

```
SELECT TRIM(agent_name) FROM agents;
```

3. Display length of policy names

```
SELECT policy_name, LEN(policy_name) FROM policies;
```

DATE FUNCTIONS

4. Display claim year

```
SELECT claim_id, YEAR(claim_date) FROM claims;
```

5. Display policies started in 2024

```
SELECT * FROM policy_assignments
WHERE YEAR(start_date) = 2024;
```

6. Display number of days between claim date and created date

```
SELECT claim_id, DATEDIFF(day, claim_date, created_at)
FROM claims;
```

AGGREGATE FUNCTIONS

7. Display total number of claims

SELECT COUNT(*) FROM claims;

8. Display average claim amount

SELECT AVG(claim_amount) FROM claims;

9. Display maximum policy premium

SELECT MAX(pre_amount) FROM policies;

GROUP BY

1. Display policy count per customer

SELECT customer_id, COUNT(policy_id)
FROM policy_assignments
GROUP BY customer_id;

2. Display claim count by claim status

SELECT claim_status, COUNT(*)
FROM claims
GROUP BY claim_status;

3. Display total premium by policy type

SELECT policy_type, SUM(pre_amount)
FROM policies
GROUP BY policy_type;

HAVING

4. Display customers having more than one policy

SELECT customer_id, COUNT(policy_id)
FROM policy_assignments
GROUP BY customer_id
HAVING COUNT(policy_id) > 1;

5. Display policy types having more than one policy

SELECT policy_type, COUNT(*)
FROM policies
GROUP BY policy_type
HAVING COUNT(*) > 1;

6. Display claim statuses with more than one claim

```
SELECT claim_status, COUNT(*)
FROM claims
GROUP BY claim_status
HAVING COUNT(*) > 1;
```

JOINS

1. Display customers with policy names (INNER JOIN)

```
SELECT c.first_name, p.policy_name
FROM insurance_customers c
INNER JOIN policy_assignments pa ON c.customer_id = pa.customer_id
INNER JOIN policies p ON pa.policy_id = p.policy_id;
```

2. Display customers with or without policies (LEFT JOIN)

```
SELECT c.first_name, pa.policy_id
FROM insurance_customers c
LEFT JOIN policy_assignments pa
ON c.customer_id = pa.customer_id;
```

3. Display policies with or without customers (RIGHT JOIN)

```
SELECT p.policy_name, pa.customer_id
FROM policy_assignments pa
RIGHT JOIN policies p
ON pa.policy_id = p.policy_id;
```

NESTED QUERIES

1. Display policies with premium above average

```
SELECT * FROM policies
WHERE pre_amount > (SELECT AVG(pre_amount) FROM policies);
```

2. Display customers who made at least one claim

```
SELECT * FROM insurance_customers
WHERE customer_id IN (
  SELECT customer_id
  FROM policy_assignments pa
  JOIN claims cl ON pa.assignment_id = cl.assignment_id
);
```

3. Display agents who sold at least one policy

```
SELECT * FROM agents
WHERE agent_id IN (
  SELECT agent_id FROM policy_assignments
);
```

CALCULATED FIELDS

1.Display premium with 6 percent tax

```
SELECT policy_name,
pre_amount,
pre_amount * 0.06 AS tax,
pre_amount * 1.06 AS total_premium
FROM policies;
```

2.Display monthly premium

```
SELECT policy_name,
pre_amount,
pre_amount / 12 AS monthly_premium
FROM policies;
```

3.Display claim amount with 5 percent processing charge

```
SELECT claim_id,
claim_amount,
claim_amount * 0.05 AS processing_fee,
claim_amount * 1.05 AS total_amount
FROM claims;
```

CASE WHEN

1.Display claim result

```
SELECT claim_id,
CASE
  WHEN claim_status = 'Approved' THEN 'Success'
  WHEN claim_status = 'Rejected' THEN 'Failed'
  ELSE 'Pending'
END
FROM claims;
```

2.Display claim priority based on amount

```
SELECT claim_id,
CASE
  WHEN claim_amount > 30000 THEN 'High'
```

```
  WHEN claim_amount BETWEEN 15000 AND 30000 THEN 'Medium'
  ELSE 'Low'
END
FROM claims;
```

3.Display policy category based on duration

```
SELECT policy_name,
CASE
  WHEN duration_year > 10 THEN 'Long Term'
  ELSE 'Short Term'
END
FROM policies;
```

ROLLUP

1.Display policy count using rollup

```
SELECT policy_type, COUNT(*)
FROM policies
GROUP BY ROLLUP(policy_type);
```

2.Display total premium by policy type using rollup

```
SELECT policy_type, SUM(pre_amount)
FROM policies
GROUP BY ROLLUP(policy_type);
```

3.Display claim count by status using rollup

```
SELECT claim_status, COUNT(*)
FROM claims
GROUP BY ROLLUP(claim_status);
```

CUBE

1.Display claim amount by customer and policy type

```
SELECT c.first_name, p.policy_type, SUM(cl.claim_amount)
FROM insurance_customers c
JOIN policy_assignments pa ON c.customer_id = pa.customer_id
JOIN policies p ON pa.policy_id = p.policy_id
JOIN claims cl ON pa.assignment_id = cl.assignment_id
GROUP BY CUBE(c.first_name, p.policy_type);
```

2.Display policy count by customer and agent using cube

```
SELECT c.first_name, a.agent_name, COUNT(pa.policy_id)
FROM insurance_customers c
```

```
JOIN policy_assignments pa ON c.customer_id = pa.customer_id
JOIN agents a ON pa.agent_id = a.agent_id
GROUP BY CUBE(c.first_name, a.agent_name);
```

3.Display total claim amount by agent and policy type

```
SELECT a.agent_name, p.policy_type, SUM(cl.claim_amount)
FROM agents a
JOIN policy_assignments pa ON a.agent_id = pa.agent_id
JOIN policies p ON pa.policy_id = p.policy_id
JOIN claims cl ON pa.assignment_id = cl.assignment_id
GROUP BY CUBE(a.agent_name, p.policy_type);
```

GROUPING SETS

1.Display policy count using grouping sets

```
SELECT customer_id, policy_id, COUNT(*)
FROM policy_assignments
GROUP BY GROUPING SETS ((customer_id), (policy_id));
```

2.Display claim count by customer and by status

```
SELECT customer_id, claim_status, COUNT(*)
FROM policy_assignments pa
JOIN claims cl ON pa.assignment_id = cl.assignment_id
GROUP BY GROUPING SETS ((customer_id), (claim_status));
```

3.Display premium summary by policy type and duration

```
SELECT policy_type, duration_year, SUM(pre_amount)
FROM policies
GROUP BY GROUPING SETS ((policy_type), (duration_year));
```

MERGE

1.Merge policy assignments into backup table

```
MERGE INTO policy_backup pb
USING policy_assignments pa
ON pb.assignment_id = pa.assignment_id
WHEN MATCHED THEN
UPDATE SET pb.end_date = pa.end_date
WHEN NOT MATCHED THEN
INSERT VALUES (pa.assignment_id, pa.customer_id, pa.policy_id);
```

2.Merge updated claims into claims_backup

```
MERGE INTO claims_backup cb
USING claims cl
ON cb.claim_id = cl.claim_id
WHEN MATCHED THEN
UPDATE SET cb.claim_status = cl.claim_status
WHEN NOT MATCHED THEN
INSERT VALUES (cl.claim_id, cl.claim_amount, cl.claim_status);
```

3.Merge new agents into agent_backup

```
MERGE INTO agent_backup ab
USING agents a
ON ab.agent_id = a.agent_id
WHEN NOT MATCHED THEN
INSERT VALUES (a.agent_id, a.agent_name, a.city);
```

CONSTRAINTS

1.Add check constraint on premium

```
ALTER TABLE policies
ADD CONSTRAINT chk_premium CHECK (pre_amount > 0);
```

2.Add not null constraint on agent name

```
ALTER TABLE agents
ALTER COLUMN agent_name VARCHAR(100) NOT NULL;
```

3.Add unique constraint on customer email

```
ALTER TABLE insurance_customers
ADD CONSTRAINT uq_email UNIQUE (email);
```

MULTI JOIN

1.Display agent, customer, policy and claim details

```
SELECT a.agent_name, c.first_name, p.policy_name, cl.claim_status
FROM agents a
JOIN policy_assignments pa ON a.agent_id = pa.agent_id
JOIN insurance_customers c ON pa.customer_id = c.customer_id
JOIN policies p ON pa.policy_id = p.policy_id
JOIN claims cl ON pa.assignment_id = cl.assignment_id;
```

2.Display agent, customer and total claim amount

```
SELECT a.agent_name, c.first_name, SUM(cl.claim_amount)
FROM agents a
```

```
JOIN policy_assignments pa ON a.agent_id = pa.agent_id
JOIN insurance_customers c ON pa.customer_id = c.customer_id
JOIN claims cl ON pa.assignment_id = cl.assignment_id
GROUP BY a.agent_name, c.first_name.
```

3.Display customer, policy and policy duration

```
SELECT c.first_name, p.policy_name, p.duration_year
FROM insurance_customers c
JOIN policy_assignments pa ON c.customer_id = pa.customer_id
JOIN policies p ON pa.policy_id = p.policy_id;
```

Joins with clauses

1. Display customer name, policy name and agent name

```
SELECT c.first_name, p.policy_name, a.agent_name
FROM insurance_customers c
JOIN policy_assignments pa ON c.customer_id = pa.customer_id
JOIN policies p ON pa.policy_id = p.policy_id
JOIN agents a ON pa.agent_id = a.agent_id;
```

2. Display customers and their policies where policy type is Health Insurance

```
SELECT c.first_name, p.policy_name
FROM insurance_customers c
JOIN policy_assignments pa ON c.customer_id = pa.customer_id
JOIN policies p ON pa.policy_id = p.policy_id
WHERE p.policy_type = 'Health Insurance';
```

3. Display agent name and number of policies sold

```
SELECT a.agent_name, COUNT(pa.policy_id) AS policy_count
FROM agents a
JOIN policy_assignments pa ON a.agent_id = pa.agent_id
JOIN policies p ON pa.policy_id = p.policy_id
GROUP BY a.agent_name;
```

4. Display customer name, policy name and claim amount

```
SELECT c.first_name, p.policy_name, cl.claim_amount
FROM insurance_customers c
JOIN policy_assignments pa ON c.customer_id = pa.customer_id
JOIN policies p ON pa.policy_id = p.policy_id
JOIN claims cl ON pa.assignment_id = cl.assignment_id;
```

5. Display customers whose claim amount is greater than 20000

```
SELECT c.first_name, cl.claim_amount
FROM insurance_customers c
JOIN policy_assignments pa ON c.customer_id = pa.customer_id
JOIN claims cl ON pa.assignment_id = cl.assignment_id
WHERE cl.claim_amount > 20000;
```

6. Display agent name and total claim amount handled

```
SELECT a.agent_name, SUM(cl.claim_amount) AS total_claim_amount
FROM agents a
JOIN policy_assignments pa ON a.agent_id = pa.agent_id
JOIN claims cl ON pa.assignment_id = cl.assignment_id
GROUP BY a.agent_name;
```

7. Display customers and policy names ordered by policy premium

```
SELECT c.first_name, p.policy_name, p.pre_amount
FROM insurance_customers c
JOIN policy_assignments pa ON c.customer_id = pa.customer_id
JOIN policies p ON pa.policy_id = p.policy_id
ORDER BY p.pre_amount DESC;
```

8. Display customer name and claim status for approved claims

```
SELECT c.first_name, cl.claim_status
FROM insurance_customers c
JOIN policy_assignments pa ON c.customer_id = pa.customer_id
JOIN claims cl ON pa.assignment_id = cl.assignment_id
WHERE cl.claim_status = 'Approved';
```

VIEWS

1. Create a view to display customer name, policy name and agent name

```
CREATE VIEW vw_customer_policy_agent AS
SELECT c.first_name, p.policy_name, a.agent_name
FROM insurance_customers c
JOIN policy_assignments pa ON c.customer_id = pa.customer_id
JOIN policies p ON pa.policy_id = p.policy_id
JOIN agents a ON pa.agent_id = a.agent_id;
```

2. Create a view to display approved claims

```
CREATE VIEW vw_approved_claims AS
SELECT claim_id, claim_amount, claim_date
FROM claims
WHERE claim_status = 'Approved';
```

3. Display records from customer policy agent view

```
SELECT * FROM vw_customer_policy_agent;
```

CHECK VIEW

4. Create a view for claims greater than 20000 with check option

```
CREATE VIEW vw_high_value_claims AS
SELECT claim_id, claim_amount, claim_status
FROM claims
WHERE claim_amount > 20000
WITH CHECK OPTION;
```

5. Insert data into high value claims view

```
INSERT INTO vw_high_value_claims
VALUES (20, 25000, 'Approved');
```

INDEXING

6. Create index on customer email

```
CREATE INDEX idx_customer_email
ON insurance_customers(email);
```

7. Create composite index on policy_assignments

```
CREATE INDEX idx_policy_customer
ON policy_assignments(customer_id, policy_id);
DROP INDEX idx_customer_email ON insurance_customers;
```

8. Drop index from insurance_customers

```
DROP INDEX idx_customer_email ON insurance_customers;
```